UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 2 **Chapter No. 2** **Lecture No. 15**

Topic : Characteristics of microservices (decentralized data management, independence), benefits (scalability, agility), and challenges (complexity, testing).

Sanjay Tiwari, E18823 **Senior Technical Trainer**

RECAP

- **The Monolithic Constraint:** While monolithic architecture is initially easy to design and implement, scalability and maintenance become severely restricted as systems grow.
- **The Microservices Paradigm:** Microservices promote a modular system design where applications are decomposed into smaller, independent services.
- **Strategic Imperative:** As emphasized in Sam Newman's Building Microservices, proper service decomposition by business capability is critical for success; poor decomposition heavily increases system complexity and operational overhead.
- **Long-Term Trajectory:** Architectural choices strongly influence long-term system growth, making microservices well-suited for modern, scalable applications.

Learning Objectives

- Characteristics of Microservices
- Benefits of Microservices
- Challenges in Microservices
- Summary and Key Insights

Overview of Microservices

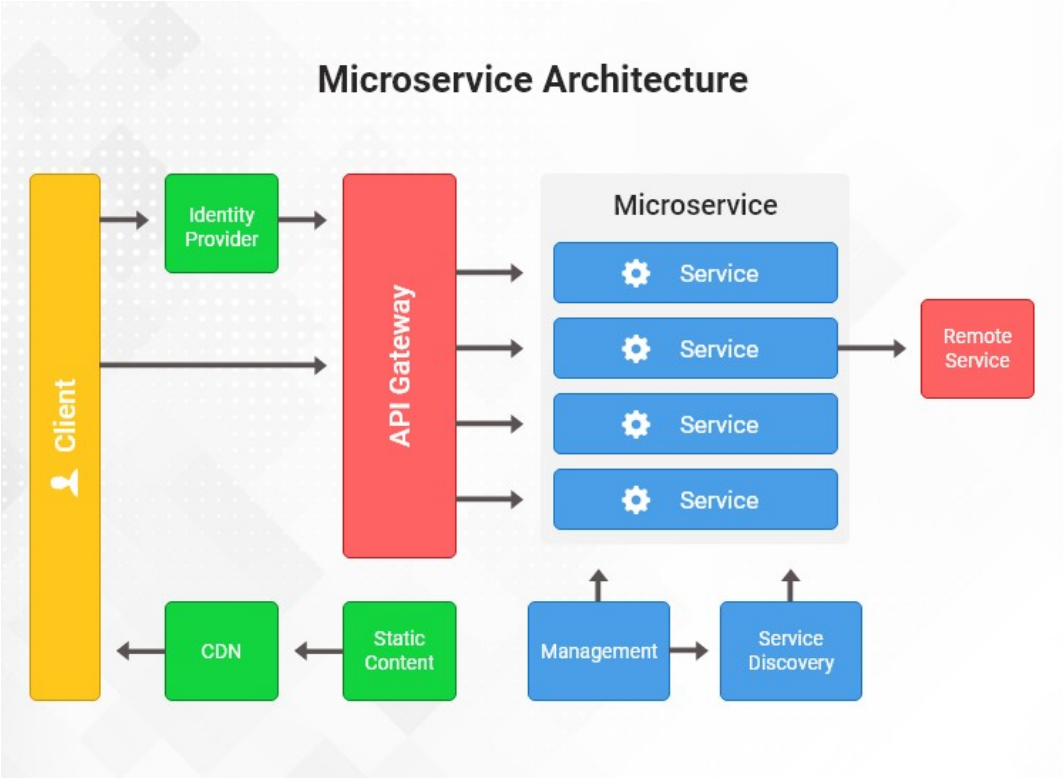
Modular Design: This architecture is based on small services where each service handles a single, specific function.

Independent Operation: These services operate independently from one another and support an independent lifecycle.

Communication: Instead of complex internal wiring, they communicate via simple, lightweight mechanisms.

Modern Standards: This modular system structure is a common, standard pattern in modern cloud-native systems.

Microservices

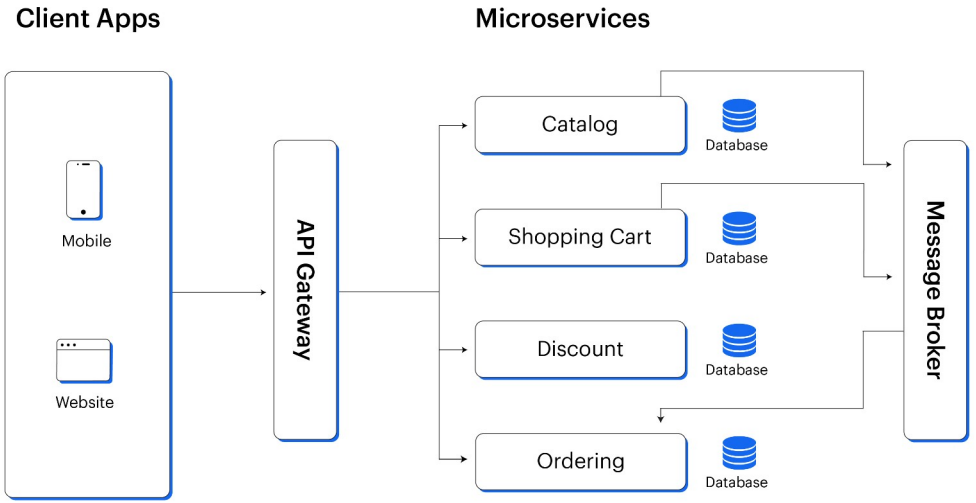


Source: Medium

Core Characteristics of Microservices

- **System Structure:** The architecture relies on loose coupling between services and high cohesion within individual services.
- **Data and Independence:** It features decentralized data management and strict service-level independence.
- **Development Flexibility:** Teams are allowed to use technology heterogeneity, meaning different services can use different programming languages or tools.
- **Operational Control:** It relies on autonomous development teams and ensures independent deployment capability.

Microservices

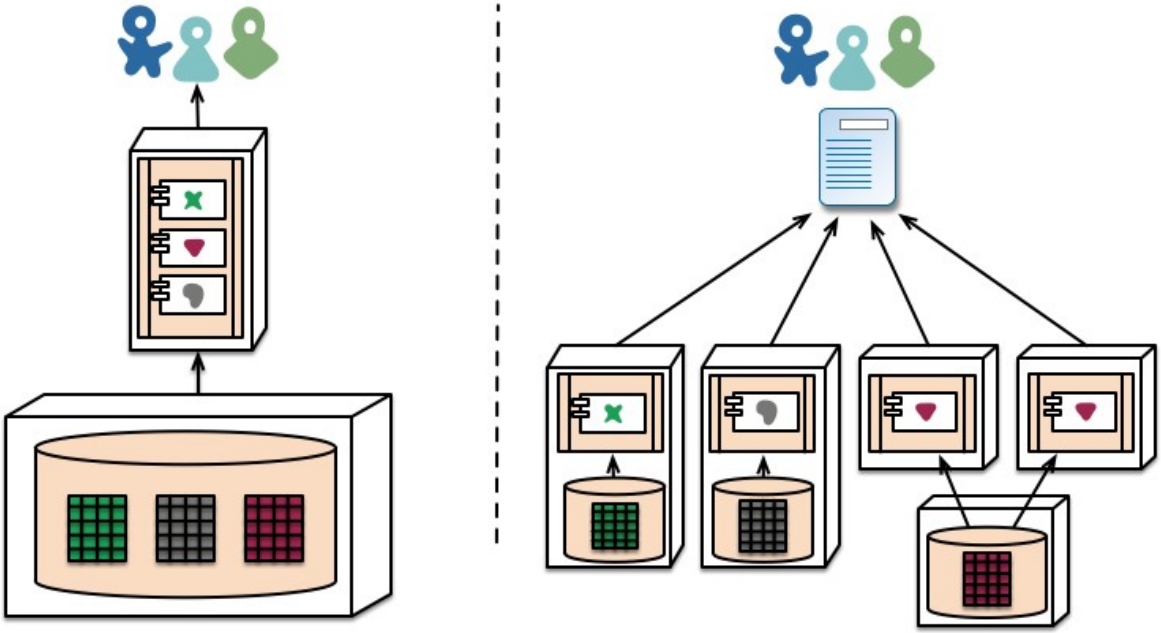


Source: Web and Crafts

Decentralized Data Management

- **The Principle of Data Encapsulation:** In a microservices architecture, each service strictly owns its data, meaning there is no shared central database across the application. Data is stored explicitly per the specific requirement of that service.
- **Decoupling the Data Tier:** This decentralized approach prevents tight coupling via the database. Consequently, it enables independent schema evolution, allowing database structures to be updated without breaking other parts of the system.
- **Resilience and Autonomy:** Distributing the data improves fault isolation. This architectural choice directly enhances service autonomy and significantly increases overall system resilience.
- **Practical Scenario:** Consider an application processing both user profiles and financial transactions. If the transaction database goes offline, the fault is isolated. The user profile service remains operational because it relies on its own decentralized data, showcasing increased system resilience.

Decentralized Data Management

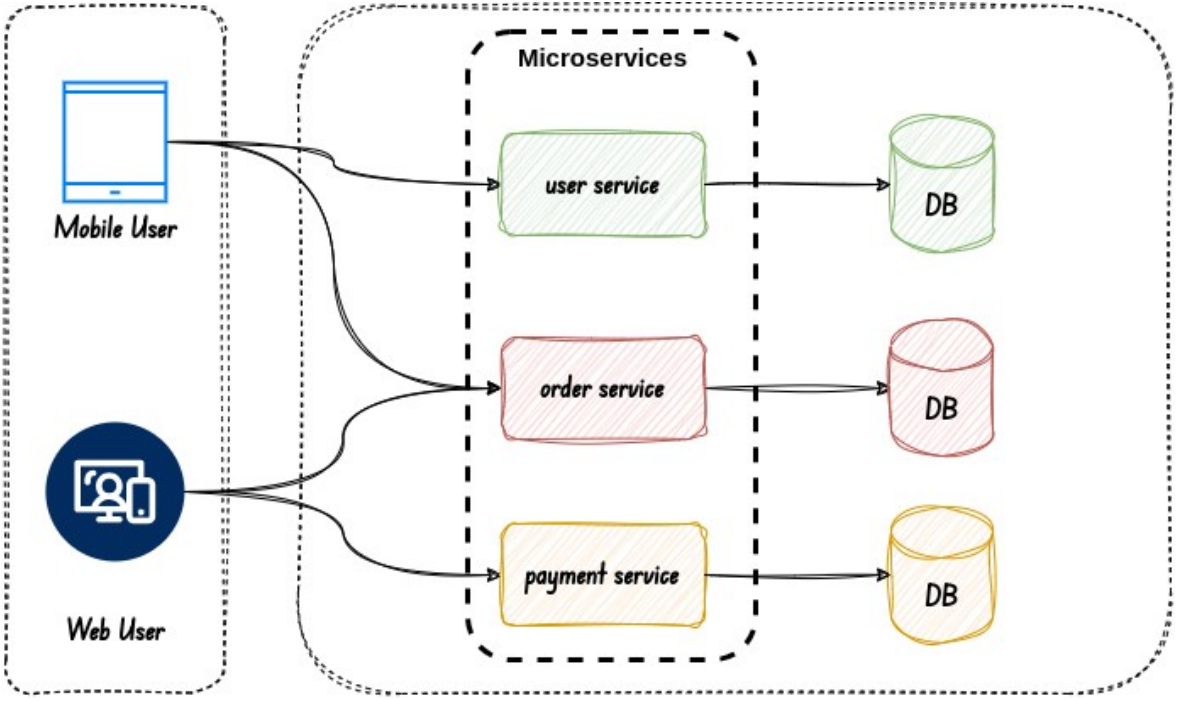


Source: Stack Exchange

Service Independence

- **Isolated Lifecycles:** Services are designed to be developed independently and can be deployed without impacting others in the ecosystem.
- **Technological Heterogeneity:** Teams are granted independent technology stack usage. This allows engineers to choose the most appropriate programming languages and tools for a specific service's function.
- **Operational Control:** Independence creates separate failure domains. Furthermore, components can be updated without causing system downtime.
- **Practical Scenario:** Two development teams can work in parallel. One team can build a feature using Python while another uses Java, demonstrating independent technology stack usage. Both teams can achieve faster development cycles and push their respective services live without system downtime.

Service Independence

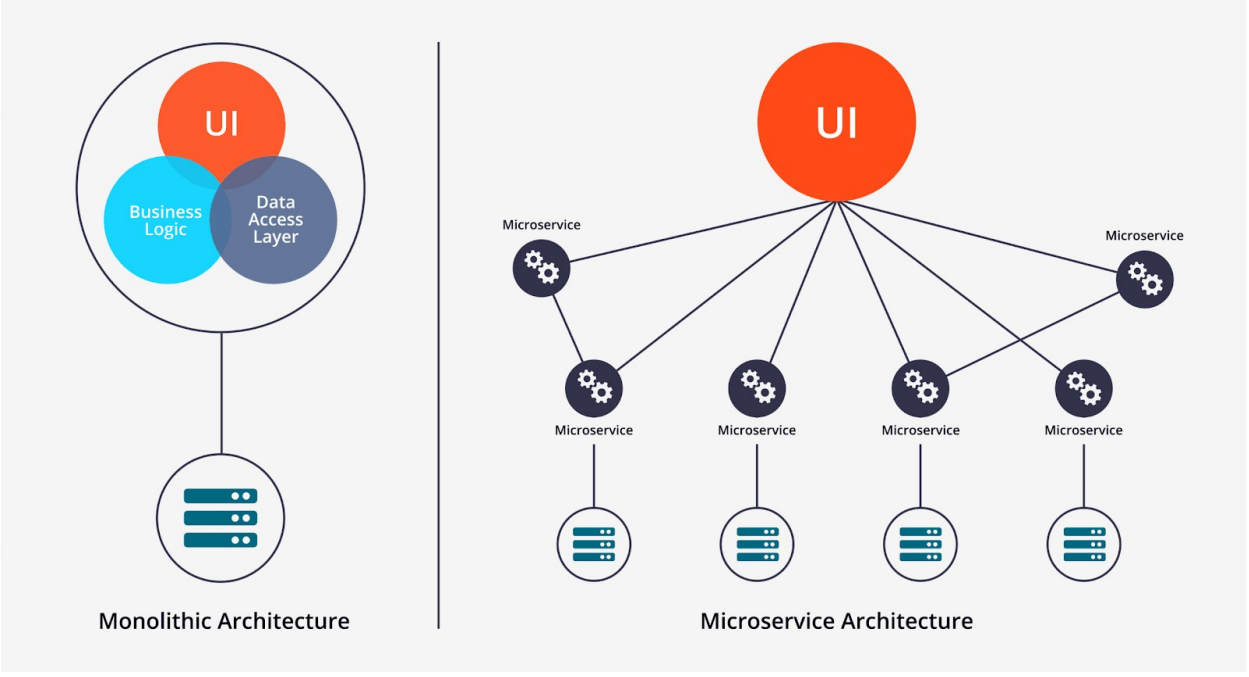


Source: AI Advances

Benefits of Microservices Architecture - Scalability

- **Granular Resource Allocation:** A primary advantage is that services scale independently. There is absolutely no need to scale the entire system when only one function is under heavy load.
- **Performance Optimization:** This model supports horizontal scaling, which handles variable workloads effectively. It inherently improves system performance through highly efficient resource consumption.
- **Cost and Growth:** Granular scaling reduces infrastructure cost and enables long-term system growth without requiring a fundamental redesign.
- **Practical Scenario:** During a high-traffic event, a specific application module may experience variable workloads. Engineers can horizontally scale only that individual service to meet demand, which is far more efficient and reduces infrastructure cost compared to replicating the entire application.

Scalability



Source: SayOne

Benefits of Microservices – Organizational Agility

- **Accelerated Delivery:** The architecture directly supports faster development and release cycles, drastically improving the product's time-to-market.
- **Empowered Engineering Teams:** By structuring the engineering department so that small teams work autonomously, there is a significantly reduced dependency between teams.
- **Adaptability to Market Needs:** This structure enables a rapid response to changing requirements, simplifying updates and fixes. It also conceptually encourages a DevOps culture and facilitates easier experimentation and innovation.

Architectural Challenges – System Complexity

- **The Cost of Distribution:** Transitioning away from monoliths leads to inherently increased system complexity. Managing distributed services introduces significant operational overhead.
- **Coordination and Governance:** Engineers face difficult service coordination due to the sheer volume of components interacting over a network. There are simply more components to manage.
- **Monitoring Burden:** This distributed nature creates complex monitoring requirements and a steep learning curve for development and operations teams. Ultimately, it requires strict, strong design discipline.
- **Practical Scenario:** When an application is split into dozens of services, tracking the health of the system requires managing distributed services simultaneously. This introduces a steep learning curve as teams must navigate complex monitoring requirements to ensure all independent parts are functioning correctly.

Operational Challenges – Testing and Debugging

- **Testing Intricacies:** Verifying system behavior is difficult because end-to-end testing becomes complex. It requires rigorous service interaction testing and careful dependency management during tests.
- **Environment Hurdles:** Engineers frequently encounter test environment setup complexity and severe data consistency issues in testing.
- **Diagnostic Difficulties:** Because requests span multiple networked services, debugging becomes a slower debugging process, and developers face harder fault reproduction. This drives an increased need for automation.
- **Practical Scenario:** If an error occurs in a distributed user flow, identifying the root cause is challenging, resulting in a slower debugging process and harder fault reproduction. Teams must manage test environment setup complexity just to simulate how multiple services interact to trigger the bug.

When to Use Microservices

- **Ideal Use Cases:** This architectural pattern is highly suitable for large and complex applications and rapidly evolving business domains.
- **Technical Prerequisites:** It is designed for systems requiring high scalability, high availability requirements, and strict modular system needs.
- **Organizational Prerequisites:** Successfully adopting microservices generally requires the presence of multiple independent development teams and a commitment to long-term product evolution within cloud-native environments.
- **Practical Scenario:** An enterprise planning for long-term product evolution in cloud-native environments is an ideal candidate.

Summary

- Microservices fundamentally emphasize independence. Implementing decentralized data improves autonomy across the system.
- Scalability and agility are major benefits, and the architecture robustly supports rapid innovation. However, system complexity is a significant challenge, and end-to-end testing requires careful planning.
- Microservices are not suitable for all applications. The architecture choice must be justified by the specific scale, complexity, and needs of the project.

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Next Lecture

Deep dive into container technology (Docker concepts), need for orchestration, introduction to Kubernetes/ECS concepts.

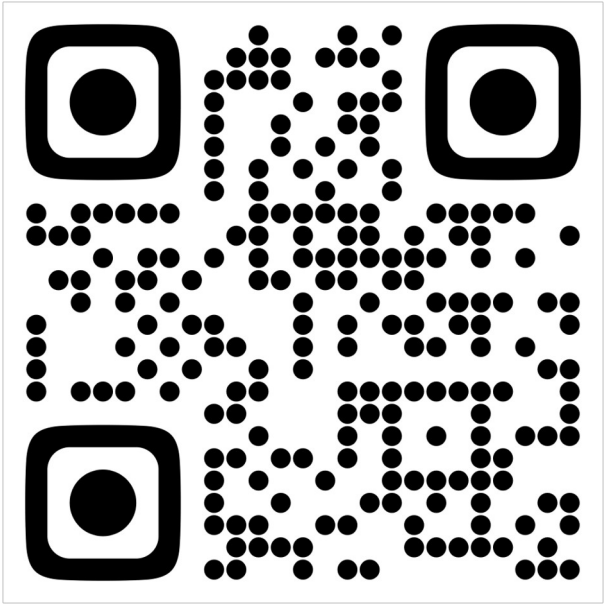
Quiz/ FAQ's

1. What is decentralized data management in microservices?
2. Why is service independence important in microservices?
3. How do microservices improve scalability?
4. What is the biggest challenge in microservices architecture?
5. Are microservices suitable for small applications?

References/ Articles/ Videos

- **Martin Fowler – Microservices Characteristics** https://martinfowler.com/articles/microservices.html
- **IBM – Microservices Architecture** https://www.ibm.com/topics/microservices
- **Microsoft – Microservices Design Principles** https://learn.microsoft.com/en-us/azure/architecture/microservices/
- **Red Hat – Benefits and Challenges of Microservices** https://www.redhat.com/en/topics/microservices
- **NIST – Software Architecture Guidance** https://www.nist.gov/itl/ssd/software-quality-group

Class-Wise Feedback



Thank You

For queries
Email: sanjay.e18823@cumail.in